

计算机视觉课程课后作业五

建智 2201 谢天赐 20221082044

5.1 缩放变换矩阵及其逆矩阵

(1) 缩放变换矩阵的逆矩阵

缩放变换矩阵用于控制图像在 x 轴和 y 轴上的缩放比例。缩放变换矩阵 S 通常表示为：

$$S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

其中 S_x 和 S_y 分别是沿 x 轴和 y 轴的缩放因子。

缩放变换矩阵的逆矩阵 S^{-1} 用于恢复原始图像尺寸，其计算方法如下：

$$S^{-1} = \begin{bmatrix} \frac{1}{s_x} & 0 \\ 0 & \frac{1}{s_y} \end{bmatrix}$$

(2) 采用逆运算进行图像缩放的方法

采用逆运算进行图像缩放，实际上是对图像进行逆缩放操作。具体步骤如下：

1. 确定缩放因子：首先确定沿 x 轴和 y 轴的缩放因子 S_x 和 S_y
2. 构造缩放变换矩阵：根据缩放因子构造缩放变换矩阵 S 。
3. 计算逆矩阵：计算缩放变换矩阵的逆矩阵 S^{-1} 。
4. 应用逆矩阵：将逆矩阵 S^{-1} 应用于图像，实现逆缩放操作。

这种方法在缩小图像时效果较好，但在放大图像时可能会出现锯齿状的边缘，因为它没有对新产生的像素点进行插值处理。

(3) 逆运算与双线性插值在图像放大时的主要区别

逆运算：直接通过逆矩阵对图像进行缩放，不涉及插值。这种方法在缩小图像时效果较好，但在放大图像时可能会出现锯齿状的边缘，因为它没有对新产生的像素点进行插值处理。

双线性插值：在放大图像时，对新产生的像素点进行插值计算，以获得更平滑的图像。这种方法可以减少锯齿状边缘，提高图像质量。双线性插值考虑了相邻像素的值，通过线性组合计算新像素的值，从而实现平滑过渡。

5.2 图像旋转的方法过程

图像旋转是一种常见的几何变换，它将图像绕某一点（通常是图像的中心）旋转一定的角度。旋转变换可以通过旋转矩阵来实现。对于角度 θ 的旋转，旋转矩阵 R 为：

$$R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

旋转矩阵的逆矩阵

旋转矩阵的逆矩阵 R^{-1} 为：

$$R^{-1} = \begin{bmatrix} \cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta) \end{bmatrix}$$

由于

$$\cos(-\theta) = \cos(\theta) \text{ 和 } \sin(-\theta) = -\sin(\theta)$$

所以逆矩阵实际上是：

$$R^{-1} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}$$

为何要采用逆运算

采用逆运算的原因主要有以下几点：

1. 恢复原始图像：如果已知旋转后的图像和旋转角度，可以通过逆旋转矩阵 R^{-1} 恢复原始图像。这对于图像处理中的某些应用（如图像校正）非常重要。
2. 处理不同方向的旋转：逆旋转矩阵 R^{-1} 可以处理顺时针和逆时针旋转，增加处理的灵活性。
3. 数学上的一致性：在数学上，任何线性变换都有其逆变换，旋转变换也不例外。使用逆变换可以保持数学上的一致性和完整性。

图像旋转的方法过程

1. 确定旋转中心和角度：首先确定图像的旋转中心（通常是图像的中心点）和旋转角度 θ 。
2. 构造旋转矩阵：根据旋转角度 θ 构造旋转矩阵 R 。
3. 应用旋转矩阵：将旋转矩阵 R 应用于图像的每个像素点，计算其旋转后的新位置。
4. 插值：由于旋转可能导致像素点落在非整数坐标上，需要使用插值方法（如最近邻插值、双线性插值等）来计算新位置的像素值。
5. 逆旋转：如果需要恢复原始图像，可以构造逆旋转矩阵 R^{-1} 并应用相同的过程。

5.3 剪切变换的编程实现

剪切变换是一种几何变换，它可以使图像沿着某一方向拉伸或压缩。剪切变换通常用两个矩阵来表示：沿 x 轴的剪切变换矩阵和沿 y 轴的剪切变换矩阵。

沿 x 方向的剪切变换矩阵：

$$S_x(\phi) = \begin{bmatrix} 1 & \tan(\phi) \\ 0 & 1 \end{bmatrix}$$

沿 Y 方向的剪切变换矩阵：

$$S_y(\phi) = \begin{bmatrix} 1 & 0 \\ \tan(\phi) & 1 \end{bmatrix}$$

其中， ϕ 是剪切角度。

任务实现：

1. 图像先沿 X 方向剪切变换 30° 、再沿 Y 方向剪切变换 30° 的结果图

首先，我们需要计算两个剪切变换矩阵的乘积，以得到先沿 X 方向剪切再沿 Y 方向剪切的总变换矩阵：

$$S_{xy} = S_y(30^\circ) \cdot S_x(30^\circ)$$

然后，将这个总变换矩阵应用于图像。

2. 图像先沿 Y 方向剪切变换 30° 、再沿 X 方向剪切变换 30° 的结果图

同样地，我们需要计算两个剪切变换矩阵的乘积，以得到先沿 Y 方向剪切再沿 X 方向剪切的总变换矩阵：

$$S_{yx} = S_x(30^\circ) \cdot S_y(30^\circ)$$

然后，将这个总变换矩阵应用于图像。

3. 可否通过一次变换实现图像沿 XY 方向同时剪切变换 ϕ 度？

理论上，可以通过矩阵乘法实现同时剪切变换，但实际上由于剪切变换的非线性特性，一次变换可能无法精确实现同时沿两个方向的剪切变换。我们可以通过计算或来验证这一点。

以 $\phi=30^\circ$ 为例，我们可以计算：

$$S_{xy} = S_y(30^\circ) \cdot S_x(30^\circ)$$

$$S_{yx} = S_x(30^\circ) \cdot S_y(30^\circ)$$

然后，像前面两题一样我们可以将这些矩阵应用于图像，并观察结果是否与先沿一个方向剪切再沿另一个方向剪切的结果相同。

代码实现：

```

# 读取图像
image = cv2.imread(r"C:\Users\86182\Desktop\1684935263zsBXkthl.jpeg")

# 定义剪切角度（以弧度为单位）
phi = np.deg2rad(30)

# 沿X方向的剪切变换矩阵，指定数据类型为 float32
Sx = np.array([[1, np.tan(phi), 0], [0, 1, 0], [0, 0, 1]], dtype=np.float32)

# 沿Y方向的剪切变换矩阵，指定数据类型为 float32
Sy = np.array([[1, 0, 0], [np.tan(phi), 1, 0], [0, 0, 1]], dtype=np.float32)

# 先沿X方向剪切再沿Y方向剪切
# 由于是仿射变换，我们可以直接将两个剪切变换矩阵相乘
S_xy = np.dot(Sy, Sx)

# 先沿Y方向剪切再沿X方向剪切
S_yx = np.dot(Sx, Sy)

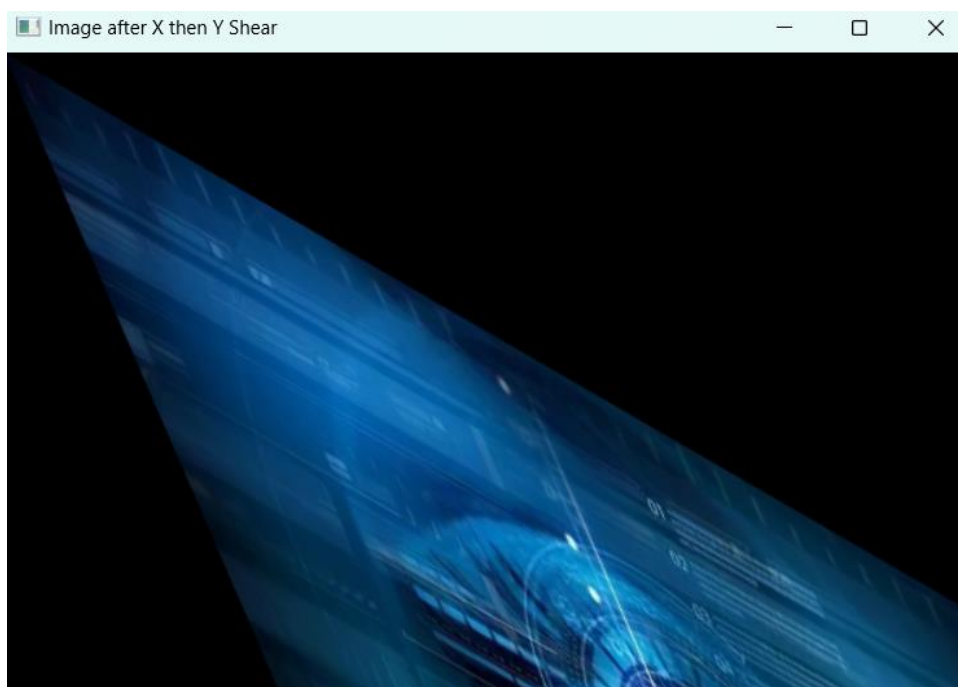
# 确保变换矩阵是 2x3 的形状
S_xy = S_xy[:2, :]
S_yx = S_yx[:2, :]

```

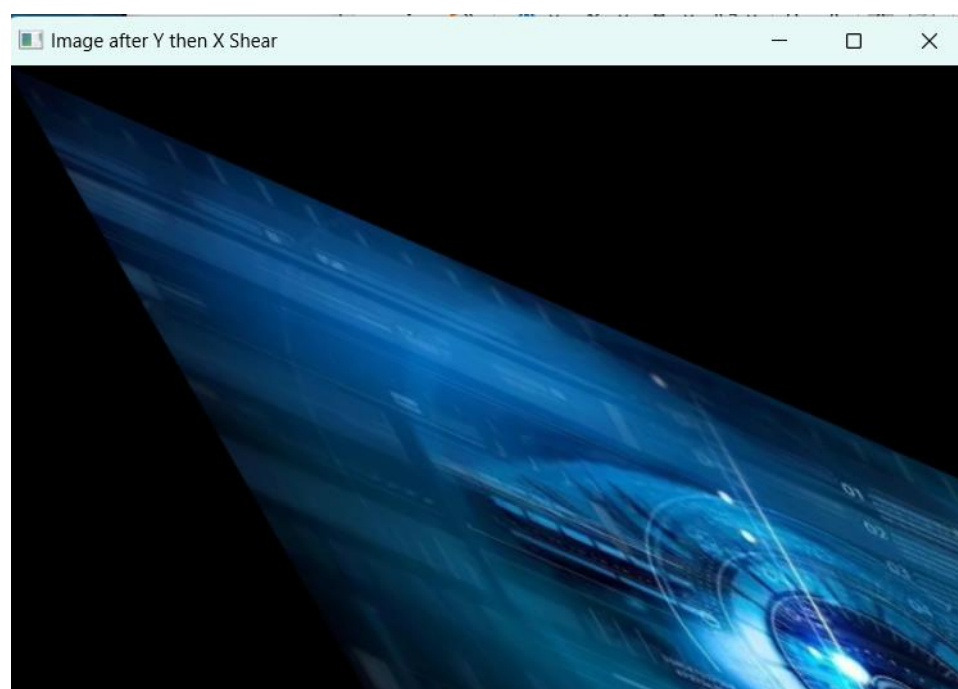
主代码如图所示。得到效果如下图



原图片



先 X 后 Y



先 Y 后 X

4. 可否通过一次变换实现图像沿 XY 方向同时剪切变换 ϕ 度?

由上题的效果图可以看出，每次剪切都会改变图像的坐标系，而这种改变是累积的，不能简单地通过一个单一的变换矩阵来实现。

